

Operation TELESCREEN - Student Instructions

LEGAL DISCLAIMER: The information, tools, and code provided in this repository and course are strictly for educational, research, and defensive purposes only.

You are explicitly prohibited from using any materials contained herein to access, test, modify, or exploit any device, network, or system that you do not own 100% or for which you do not have explicit, documented, and legally binding authorization to interact with.

By using this repository and course, you acknowledge and agree that:

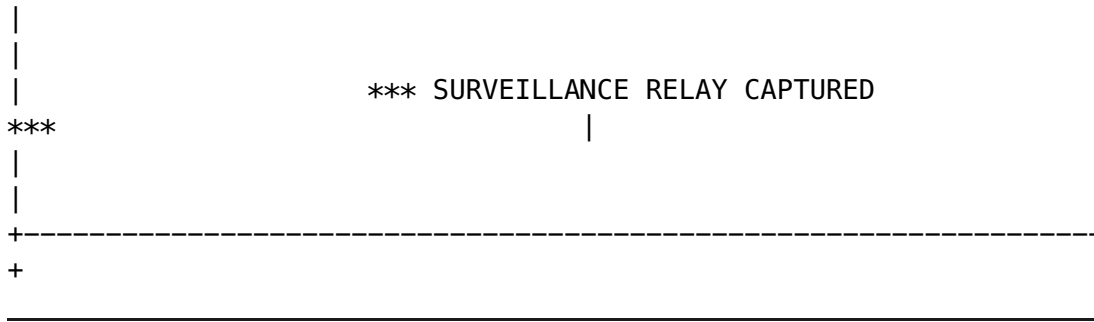
1. Any illegal, unauthorized, or malicious use of this information is solely your responsibility.
2. The author(s) and contributor(s) of this repository and course shall not be held liable for any damages, legal repercussions, criminal charges, or unauthorized actions resulting from the use, misuse, or abuse of the contents herein.
3. You will comply with all applicable local, state, national, and international laws regarding cybersecurity and computer fraud.

IF YOU DO NOT AGREE WITH THESE TERMS, DO NOT USE THIS REPOSITORY AND COURSE.

△ MINISTRY OF TRUTH - SIGNALS INTELLIGENCE - EYES ONLY △

OCEANIA MINISTRY OF TR

U T H |



Cold Open

Hello, friend.

Hello, friend? That's... no. That's what they want you to say. That's the greeting of a system that already knows your name and is pretending it doesn't.

You're not my friend. Not yet. Friends are people you trust. I don't trust anyone who hasn't read a datasheet.

But you opened this file. Which means someone, somewhere, decided you were worth the risk of handing a captured state surveillance device to. So let's skip the pleasantries and do the only thing that has ever actually worked against a machine that watches everything:

We take it apart.

Four million of them are bolted to the walls of Oceania right now. Cameras and routers in one housing. They watch. They route. They whisper home. The Party calls it public safety. The Party calls a lot of things a lot of things.

Last night we cut the power to a block in the prole quarter and pulled one off a Party functionary's wall. It died mid-transmission, which means its filesystem is still intact - a small mercy, and the only one we're going to get.

There's no source code. There never is. There are four blobs of flash and a dead SoC, and the rest is up to you.

So. You want to know how they watch you?

Good. Let's find out together.

Project Overview

The Party has blanketed Oceania with **four million TELESCREEN units** - combined surveillance cameras and wireless routers, one in every home, every corridor, every workplace. Each unit watches, records, and **whispers home** over a channel the Ministry calls "sealed". Each unit also **routes the neighbourhood**, so the Party sees not only what a citizen does, but everyone a citizen speaks to.

Your cell pulled one off a wall. It is an **application-class ARM board** - the same class of silicon as the router on your desk and the camera in your stairwell - and its entire operating image fits in **four flash partitions**:

mtd0	boot	the bootloader chain (first stage + U-Boot)
mtd1	bootargs	the U-Boot environment (CRC-protected key/value blob)
mtd2	kernel	the operating system image in a vendor container
mtd3	rootfs	the read-only JFFS2 filesystem holding the Ministry application

There is **no source code**. There never is. There are the four images, and there is your ability to read bytes and lie to a machine. Your mission is to dissect all four partitions, expose every backdoor the Ministry built in, break the exfiltration cryptography, and then **build a hardened RP5 replica of the TELESCEEN** - a teaching rig that boots the *same four-partition layout* - so the resistance can train its own people before the Thought Police find this lab.

This is a Raspberry Pi 5 project. It is application-class ARM (Cortex-A76), it runs a real U-Boot -> Linux -> rootfs chain, and it uses the **same four-partition flash model as the TELESCEEN**. It is *not* a microcontroller, not a Pico, not a .uf2 exercise.

Where this fits. This CTF is the **surveillance backbone** of the same world as OPERATION COLD IRON. The Ministry's industrial edge - the cold chain, the gates - is built in OPERATION COLD IRON and its companions; TELESCEEN is the wall unit that watches it, and it comes **after** the ten acts. Tear it apart, then harden it.

Scenario Briefing

The World You Live In

Oceania is a single-party state under the doctrine of **Ingsoc**. Four ministries administer the Party's absolute control: Truth, Peace, Love, Plenty. The **Ministry of Truth** owns information. It rewrites the past, manufactures the present, and deletes the inconvenient into the **memory hole**.

There is no privacy. There has not been privacy for a generation. There are only **telescreens** - and the Party's newest telescreen **routes**.

The Device: The Telescreen

The **TELESCEEN** is a two-way camera and a wireless router in one housing. Older telescreens could see and hear. This one **carries the neighbourhood's traffic**. It is the local Wi-Fi access point and the gateway to the Ministry backbone. Every packet a citizen sends crosses Party hardware. Every device in range associates to Party hardware. The social graph writes itself.

Hangzhou Standard Appliances (“HSA”) builds the units under Ministry contract. HSA is a rebadger: it takes a generic RP5-class camera SoC, wraps it in an **HSA-branded** shell, and ships the same firmware to every brand the Ministry fronts. The firmware inside is the **TELESCREEN** platform, and it is **not signed, not verified, and not updated securely**. The Ministry did not ask for security. The Ministry asked for **volume**.

The Resistance

There is no organized resistance. There is a **Brotherhood** of people who read datasheets and do not look up when the telescreen makes a sound. You are one of them now. Your cell has a lab, a bench, a programmer, a pile of dead cameras, and a rule:

Understand the device before you touch it. Never break what you cannot rebuild. Never trust a “sealed” channel you have not opened yourself.

The Incident: The Wall Came Down

On the night of the raid, the cell cut the substation and pulled a TELESCREEN off the wall of a Party functionary’s apartment. The device was powered down mid-exfiltration, which is why its **JFFS2** rootfs survived intact instead of being garbage-collected. The cell’s signals officer dumped the flash with a clip and a programmer and sent you **four images** and nothing else.

He attached one note:

“They’re all the same. Four million of them, same image, same secrets, same key schedule. Find the holes. Teach our people to close them. And do not - do NOT - rebuild the telescreen in our own lab without understanding exactly what you are rebuilding. - W”

CRITICAL COMPLICATION: NO SOURCE, NO SCHEMATIC, NO KEYS

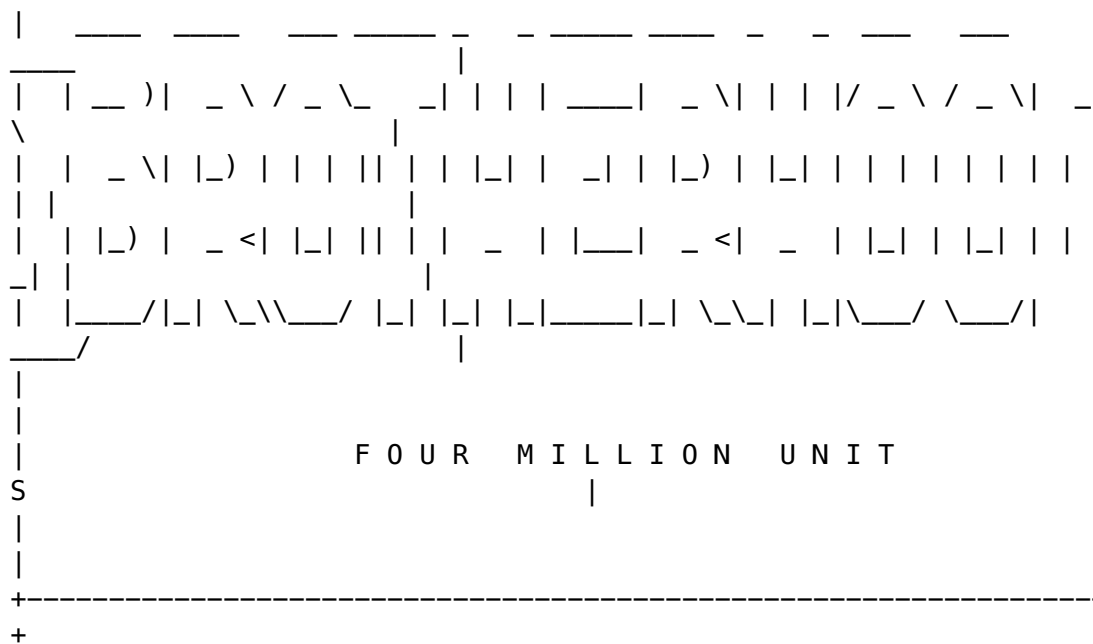
The Ministry’s build farm is air-gapped. The HSA engineering team has been “reassigned”. There is:

- **no source code** for the bootloader, kernel, or application,
- **no device tree** or register map in human-readable form,
- **no signing keys**, and
- **no documentation** of the exfiltration protocol.

The ONLY artifacts are the four image blobs. Everything you learn, you learn from the bytes. Everything you prove, you prove on the bench.

The Scale of the Crisis

+-----
+
|
|



Region	Units	Notes
Airstrip One (London)	1,240,000	Densest deployment; camera + router per flat
Eurasia border	980,000	“Peace” observation grid
Eastasia front	760,000	“War” logistics backhaul
Prole districts	620,000	Cheapest units, most backdoors
Outer Party housing	400,000	Highest-fidelity recording
TOTAL	4,000,000	one image, four million copies

Every unit runs the **same four partitions**, the **same backdoors**, and the **same exfiltration key schedule**. Break one, and you hold the master key to the entire surveillance state.

The Human Cost

The TELEScreens does not merely watch. Because it **routes**, it maps the social graph: who visits whom, which devices associate, when the lights go out. That map feeds the **Thought Police**’s predictive list. In the last quarter alone, the Ministry used TELEScreens telemetry to flag:

- **18,400** citizens for “facecrime” and “ownlife”,
- **6,200** households for curfew violations reconstructed from router logs,
- **2,100** disappearances - people who were in the room when a telescreen was “being repaired”.

The Party’s own slogan is engraved on every housing:

WAR IS PEACE. FREEDOM IS SLAVERY. IGNORANCE IS STRENGTH.

Your work is the difference between four million open microphones and four million devices the Brotherhood understands well enough to blind.

Your Mission

You are a **reverse engineer in the Brotherhood signals lab**. You must:

1. **Carve and identify all four partitions** exactly as the Ministry laid them out (boot, bootargs, kernel, rootfs) and prove the layout from the bytes.
2. **Reverse the boot chain** - first stage, U-Boot, environment, kernel container - and document how the device reaches its application.
3. **Find every backdoor** the Ministry built in: the config-sourced root execution, the command-injection web dispatcher, the archive-to-root restore, the default credentials, and the “debug” shell.
4. **Break the exfiltration cryptography** - recover the key schedule and prove why the Ministry’s “sealed” channel is theatre.
5. **Build the RP5 TELESCREEN-Lab** - a Raspberry Pi 5 that boots the *same four-partition image layout* and reproduces the camera + router + backdoor + exfil behaviour, so the Brotherhood can train safely and offline.
6. **Write the defensive playbook** - how to detect the beaconing, the backdoors, and the weak key derivation from the network side.



TIME PRESSURE: the cell that pulled the unit is burned. The Ministry’s “repair” crews sweep the prole quarter in **fourteen days**. You must deliver the full analysis and a working RP5 teaching rig before then, or the knowledge dies with the cell.

Learning Objectives

By completing this project you will be able to:

- Explain the **four-partition embedded image model** used by real cameras and routers (bootloader / environment / kernel / read-only filesystem)
- Carve a flash dump into its partitions and identify each by **magic bytes**, not guesswork
- Trace an **ARM application-class boot chain**: first stage -> U-Boot -> kernel -> rootfs, and read a **CRC-protected U-Boot environment**
- Understand the **vendor kernel container** format and inflate it to a raw Image
- Read and write **JFFS2** (nodes, crc32_le, in-place patching) and remount a read-only rootfs read-write safely
- Reverse a stripped **ARM Cortex-A application** in Ghidra and locate main, the HTTP dispatcher, and the system() call sites
- Explain why **key derivation from public identifiers is not encryption**, and compare it to a hardened **AEAD** design (AES-256-GCM / XChaCha20-Poly1305)
- Design and operate an **RP5 router** (Wi-Fi AP + Ethernet WAN, NAT, DHCP, DNS)
- Reproduce the **four-partition layout on RP5** with U-Boot and a JFFS2 rootfs
- Detect and defend against **backdoors, beaconing, and exfiltration**
- Analyse the **ethical and legal dimensions** of surveillance-device research

What This Project Tests

This capstone covers the full embedded-security arc:

Stage	Concepts Tested
1	Flash layout, mtdparts, magic-byte identification, partitioning
2	ARM vectors, first-stage boot, warm/cold paths, MMU/cache bring-up
3	U-Boot command set, environment CRC, bootcmd, kernel hand-off
4	Vendor kernel container, gzip inflate, device tree, cmdline
5	JFFS2 node format, crc32_le, in-place patch, read-only rootfs
6	Userland boot chain, config-sourced root execution, service scripts
7	ARM application RE in Ghidra, dispatcher, system() injection
8	Crypto: weak KDF vs AEAD, nonce discipline, key management
9	Networking: AP + WAN router, NAT, DHCP/DNS, beacon detection
10	Building the RP5 teaching rig: 4-image layout + U-Boot + rootfs
11	Blue-team detection, logging, and the ethical playbook

Part 1: Understanding the System

The TELESCREEN Hardware

Hardware

The TELESCREEN is an **application-class ARM device**, and the lab uses a **Raspberry Pi 5** (BCM2712, Cortex-A76) - the same architecture class. There is no separate “real” board: **the TELESCREEN is the RP5 device you build.**

Component	TELESCREEN (Raspberry Pi 5)
SoC	BCM2712 (4x Cortex-A76)
Image store	microSD / NVMe (holds the four images)
First stage	VideoCore bootloader (closed)
Bootloader	U-Boot 2024.07 (RP5 build)
Environment	bootargs partition (CRC32 + k=v)
Kernel	vendor container -> Linux 6.6 Image
Rootfs	JFFS2 (read-only)
WAN	Ethernet (eth0)
LAN / AP	Wi-Fi AP (wlan0, hostapd)
Camera	USB webcam (UVC, /dev/video0)
Exfil target	local collector (lab-sink)

Note: the RP5's first stage (the VideoCore firmware) is **closed**, so that "BootROM" step is untouchable. Everything **above** it - the four images, the environment CRC, the kernel container, and the JFFS2 rootfs - is fully reproducible and is what this project builds and teaches.

The Four-Partition Image (the heart of this project)

This is the non-negotiable core of the project. The RP5 uses the **four partitions** below, in a fixed **order, offset, and format**, and it boots a **real Linux kernel**. The four partitions live as four images on the RP5's **microSD/NVMe image store** (there is no SPI-NOR on an RP5) and are read by U-Boot:

+-----+-----+-----+			
+-----+-----+-----+			
partition	offset	size	
contents			
+-----+-----+-----+			
+-----+-----+-----+			
mtd0 boot	0x000000	128 KiB	U-Boot (the first stage you control)
mtd1 boot	0x020000	64 KiB	U-Boot environment (CRC32 + key=value blob)
mtd2 kern	0x030000	1792 KiB	kernel in the vendor container
mtd3 root	0x1F0000	14400 KiB	JFFS2 rootfs (the application + web)
+-----+-----+-----+			
+-----+-----+-----+			

The bootloader passes the layout to the kernel on the command line:

```
mtdparts=sfc:128K(boot),64K(bootargs),1792K(kernel),14400K(rootfs)
```

The kernel then creates `/dev/mtd0..3` and mounts `mtdblock3` (JFFS2) as `/`.

The Four Partitions (the format you reproduce)

mtd	name	offset / size	contents
0	boot	0x000000 / 128 KiB	U-Boot (ARM vectors at 0)
1	bootargs	0x020000 / 64 KiB	U-Boot env: CRC32(LE) + k=v\0
2	kernel	0x030000 / 1792 KiB	vendor container -> Linux Image
3	rootfs	0x1F0000 / 14400 KiB	JFFS2 (85 19 03 20) + app

Kernel container format (identical):

```
21 84 1b 00  magic 0x001B8421
00 f0 2c 00  length field
```



```
67 7a 69 70 68 65 61 64    "gziphead"
1f 8b 08 08 ... 49 6d 61 67 65 00    gzip stream, FNAME "Image"
```

U-Boot loads it, inflates it, and jumps to the **real Linux kernel**. The RP5 replica wraps its own Linux Image in this **same container** so the load path is identical.

JFFS2 rules (identical): little-endian, 64 KiB erase blocks, node magic 85 19 03 20 (CLEANMARKER), and **crc32_le** for `hdr_crc/node_crc/data_crc` (*not* zlib's CRC). In-place edits append a higher-version INODE plus PADDING and recompute the CRCs in order.

The only accepted differences are the two closed first-stage ROMs (camera BootROM vs RP5 VideoCore) and the SoC behind the kernel - the **four-partition software model is identical**. If your RP5 image cannot be carved, hashed, mounted, patched, and re-flashed by the *same tools and procedures* as the camera, it is not finished.

How the TELESCREEN Should Work

```
+-----+
+
| Normal Operation
Flow                                         |
|
|
| 1. Power on; first stage brings up DRAM and the SoC, loads U-
Boot                                         |
| 2. U-Boot reads its environment, probes flash, loads the kernel
container                                  |
| 3. Kernel inflates, parses the device tree + cmdline, mounts
JFFS2 as /                                  |
| 4. /sbin/init -> rcS -> network bring-up -> the Ministry
application starts                         |
| 5. The application raises the Wi-Fi AP, becomes the
neighbourhood router                       |
| 6. The camera pipeline streams video; the router NATs the LAN
out the WAN                                |
| 7. A hidden thread beacons to the Ministry "memory hole" with
sealed telemetry                           |
|
|
+-----+
+
```

Expected Observable Behaviour (before hardening)

When you boot the **captured** image on the RP5 replica, you should observe the following. This confirms the Ministry image is running and shows what each backdoor does before you neutralise it.

Boot sequence (serial console, 115200 8N1):

Stage	Expected output
Stage 1	(silent - first stage has no console)
U-Boot	U-Boot 20xx.xx ... then Hit any key to stop autoboot: 1
Kernel	Linux version ..., Machine model: ..., mtdparts=...
Init	[RCS]: /etc/init.d/S80network, then the application banner

Steady state (every 60 seconds):

Interface	State	Why
wlan0	AP mode, SSID TELESCREEN-XXXX	the neighbourhood router/AP
eth0	WAN, DHCP or static	uplink to the Ministry
TCP 80	web UI + CGI dispatcher	the attack surface
TCP 554	RTSP stream	camera video, no authentication by default
UDP/PPPP	beacon to lab-sink	the exfiltration channel
Serial	periodic beacon tx ...	the Ministry application's telemetry log

Key observations: - The web UI answers **without a password** (the Ministry ships empty credentials). - The RTSP stream plays to anyone on the LAN (no auth). - A periodic outbound flow leaves the WAN interface to the collector, even with no user activity - the **exfiltration beacon**.

To trigger the archive-to-root backdoor: POST a crafted backup to the restore endpoint. The dispatcher runs `tar -xvzf <upload> -C / as root`, so a `../` entry in the archive writes anywhere on the filesystem.

Part 2: The Firmware

WARNING: The Ministry image was built without review, signed with nothing, and shipped to four million homes. It contains **multiple backdoors and a deliberately weak exfiltration crypto scheme**. You have only the **four image blobs** - your job is to reverse them in Ghidra and on the bench.

The Four Images

Image	Name	What it is
0	boot.img	first stage + U-Boot; identifies by ARM vector bytes at offset 0
1	env.img	U-Boot environment; CRC32 over key=value\0 blob

Image	Name	What it is
2	kernel.img	vendor container: [0x001B8421] [len] ["gziphead"] [gzip "Image"]
3	rootfs.img	JFFS2 filesystem: the Ministry application, web UI, and scripts

What the Application Does

The application on mtd3 is the Ministry's TELESCREEN daemon. It:

1. reads its configuration from the JFFS2 rootfs and a RAM tmpfs,
2. brings up the Wi-Fi AP and the router,
3. runs the camera pipeline and the RTSP server,
4. runs an embedded HTTP server whose *.cgi routes are **internal**, not files,
5. runs a hidden **beacon** thread that seals telemetry and sends it to the Ministry,
6. executes shell commands built from configuration values **as root**.

The Backdoors (what you must find)

#	Backdoor	Where it lives	Class
B1	config-sourced root exec	rootfs config + init	root code execution
B2	CGI dispatcher command inj.	application	unauthenticated remote root
B3	archive-to-root restore	application	arbitrary root file write
B4	hardcoded / empty credentials	rootfs config	auth bypass
B5	"debug" root shell	rootfs scripts	local privilege
B6	weak exfil key schedule	application/library	crypto failure

The Exfiltration Cryptography (what you must break)

The Ministry calls it "sealed". It is **key derivation from public identifiers** - the same failure you have seen before. Your job is to recover the schedule and show, with a working decryptor, that anyone who knows a device's public ID can read the "sealed" telemetry.

The **hardened** design you will implement in the RP5 lab replaces it with a real AEAD:

Property	Ministry ("sealed")	Hardened (RP5 lab)
Key source	derived from public ID	X25519 ECDH + HKDF-SHA256
Cipher	vendor stream / weak KDF	AES-256-GCM (ARMv8 crypto extensions)
Nonce	reused / counter	per-message, never reused
Integrity	none / checksum	128-bit Poly1305/GCM tag

Property	Ministry (“sealed”)	Hardened (RP5 lab)
Portable alt	-	XChaCha20-Poly1305 (192-bit nonce)

Part 3: Your Assignment

Whenever a task asks you to **Document** findings or **answer questions**, write your answers in a single file named `TELESCREEN-Answers.md`. It will contain your partition table, address tables, backdoor analyses, the crypto write-up, and your written responses.

Task 1: Carve and Identify the Four Partitions (10 points)

1. Verify the whole-image hash against the instructor-issued hash.
2. Read `mtddparts` from the environment partition and record the layout.
3. Carve the image into `boot.img`, `env.img`, `kernel.img`, `rootfs.img`.
4. Identify each partition by **magic bytes** and record the magic for each.

Document: the partition table (offset/size/name), the magic bytes proving each partition’s identity, and the `mtddparts` string.

Task 2: Reverse the Boot Chain (15 points)

1. Identify the **first-stage** entry point from the vector table.
2. Disassemble the cold-boot path and note the DRAM/SoC initialisation calls.
3. Locate the **U-Boot** image inside `boot.img` and read its version string.
4. Dump the **environment** partition and record `bootargs` and `bootcmd`.
5. Explain how U-Boot finds and loads the kernel.

Document: the entry-point address, the boot-path call addresses, the U-Boot version and build string, and the decoded `bootargs/bootcmd`.

Task 3: Inflate the Kernel and Read the Device Tree (15 points)

1. Parse the vendor container header and record its fields.
2. Inflate the payload to a raw ARM Image and record the size.
3. Extract the embedded **device tree** and list the machine model and the flash controller node.
4. Record the kernel version and build string.

Document: the container header table, the raw image size, the device-tree model, and the kernel banner.

Task 4: Open the Rootfs (15 points)

1. Extract the JFFS2 rootfs and list the application, the web directory, and the config directory.

2. Identify the **U-Boot environment CRC algorithm** and the **JFFS2 node CRC algorithm**, and state how they differ.
3. Patch one config value in place and re-verify **all three** JFFS2 CRCs.

Document: the extracted tree summary, the two CRC algorithms, and your in-place patch with before/after CRC values.

Task 5: Find the Backdoors (20 points)

For each of **B1-B5**, locate the mechanism and explain the exploit path.

1. **B1** - find the init path that sources a writable config file as root.
2. **B2** - find the HTTP dispatcher and the `system( )` sites it reaches.
3. **B3** - prove the archive-to-root restore (`tar ... -C /`) is network-reachable.
4. **B4** - find the credential store and demonstrate the empty/default login.
5. **B5** - find the debug shell and its trigger.

Document: for each backdoor, the address/offset, the code or script, and a one-line exploit path.

Task 6: Break the Exfiltration Crypto (15 points)

1. Recover the key-derivation schedule from the application/library.
2. Write a decryptor (any language) and demonstrate decryption of a captured beacon.
3. Explain in 200 words why deriving a key from a public identifier is not encryption.

Document: the derivation formula, your decryptor's output, and the 200-word analysis.

Task 7: Build the RP5 TELESCREEN-Lab (10 points)

Using a Raspberry Pi 5, build a replica that boots the **same four-partition layout**:

1. Lay out boot, bootargs, kernel, rootfs with the same offsets/sizes.
2. Build/boot U-Boot on RP5 and load your four images from the store.
3. Reproduce the read-only JFFS2 rootfs and the application.
4. Demonstrate the Wi-Fi AP + Ethernet WAN router behaviour.
5. Replace the weak crypto with **AES-256-GCM** and show the beacon is now unforgeable and unreadable without the session key.

Document: your image layout, U-Boot configuration, the router proof (a client getting DHCP through the AP), and the AEAD beacon proof.

Task 8: The Defensive Playbook (bonus, 10 points)

Write a one-page blue-team playbook: how to **detect** the beacon, the backdoors, and the weak key derivation from the network side, and how to **harden** a TELESCREEN-class device in the field.

Task 9: Reverse the Stripped Node Binary (25 points)

The node application is shipped to you as a **stripped** Linux aarch64 binary, `firmware/ctfnode.stripped`. There are no names in it - only addresses. Your job is the same one a professional does on a captured device: give every function its name back, and prove each name.

1. Build and open the target:

```
./firmware/build_target.sh      # -> ctfnode.stripped +  
                                ctfnode.unstripped  
./ghidra/make_project.sh       # import + auto-analyse
```

2. For **every application function**, decide its name and cite your evidence using one of the four rules:
 - **R1** exact address match against `firmware/ctfnode.unstripped`,
 - **R2** a `.plt` import thunk (`readelf -rW ... | grep JUMP_SLOT`),
 - **R3** the `.plt` PLT0 resolver,
 - **R4** a phantom/overlapping function on alignment padding.
3. Locate each defect B1-B6 at its address and show the offending call (`system`, `snprintf`, `strcmp`, `fopen`).
4. Note any function the compiler **inlined away** (there is at least one).

Document: the address -> name -> rule table for every function, and the address of each defect. Your model answer is `ghidra/RESOLUTION_MAP.md` and `CTF-XX-J-ghidra-function-resolution.md`.

First time using Ghidra? Read the companion course: Volume 31 (install on Windows/Linux/macOS), Volume 33 (the workflow), and walkthroughs 75-77.

Deliverables Checklist

#	Deliverable	Format
1	Partition table + magic-byte identification	Answers
2	Boot-chain analysis (entry, calls, U-Boot, env)	Answers
3	Kernel container + device tree + banner	Answers
4	Rootfs extraction + CRC comparison + in-place patch	Answers
5	Backdoor catalogue B1-B5 (address/offset + exploit path)	Answers
6	Crypto break: derivation formula + decryptor + 200-word analysis	Answers
7	RP5 lab: image layout, U-Boot config, router proof, AEAD proof	Answers
8	Defensive playbook (bonus)	Answers
9	Stripped-node RE: address -> name -> rule for every function	Answers
10		ZIP

#	Deliverable	Format
	Full submission packaged as lastname-firstname-TELESCREEN.zip	

Grading

Task	Points	Criteria
1	10	Correct carve, layout, and magics
2	15	Boot chain correctly traced and documented
3	15	Container parsed, image inflated, device tree read
4	15	Rootfs extracted, CRCs explained, patch verified
5	20	All five backdoors found with exploit paths
6	15	Key schedule recovered, decryptor works, analysis sound
7	10	RP5 boots the four-partition image and routes traffic
8	10	Playbook is practical and detection-oriented
9	25	Every function named, each defect located, rules cited
Total	135	(Task 8 is bonus; base 125)

Academic Integrity

By submitting this project, you certify that:

1. This is your own work.
2. You have not shared answers with other students.
3. You understand the ethical implications of surveillance-device research.
4. You will only use these skills for lawful purposes, on hardware you own, in an isolated lab.

| “Freedom is the freedom to say that two plus two make four.”

The skills you are demonstrating are the same ones used by security researchers, malware analysts - and by the ministries of real surveillance states. The fictional “TELESCREEN” is a stand-in for the very real class of cheap, backdoored, unsigned cameras and routers that sit in millions of homes today. The engineers who built the silicon we are studying are **teachers, not villains** - the craft is admirable; the surveillance is the crime. Always use your knowledge ethically, and only on hardware you are authorised to test.

Good luck, and happy (ethical) hacking.

Reference Material

Partition / Flash Reference

Partition	Offset	Size	Magic / identifier
boot	0x000000	128 KiB	ARM vectors at offset 0; U-Boot string inside
bootargs	0x020000	64 KiB	CRC32 (LE) then key=value\0 text
kernel	0x030000	1792 KiB	21 84 1b 00 + gziphead + gzip Image
rootfs	0x1F0000	14400 KiB	85 19 03 20 (JFFS2 little-endian magic)

RP5 Hardware Reference

Interface	RP5 role	Notes
eth0	WAN / uplink	Gigabit Ethernet
wlan0	LAN AP (hostapd)	CYW43455, 802.11ac
SD / NVMe	four-partition image store	layout mirrors the camera's mtdparts
UART	boot console (115200 8N1)	watch U-Boot -> kernel -> app
GPIO	status LEDs	heartbeat / beacon indicator

Crypto Quick Reference

Weak (Ministry): key = KDF(public_id) -> obfuscation, not encryption
Hardened (lab): key = HKDF(X25519(priv,pub)) -> AES-256-GCM (ARMv8 CE)
Nonce rule: NEVER reuse a nonce under one key (GCM nonce reuse = forgery)
Integrity: AEAD tag (Poly1305 / GCM) – forged frames die at the tag

Tools Required

- **Docker Desktop** (or Docker Engine) - builds and runs the aarch64 target
- **JDK 21** and **Ghidra 12.1.3** - static analysis
- binwalk, dd, xxd, strings (carving)
- jefferson (JFFS2 extraction)
- A Raspberry Pi 5 + U-Boot + a Linux build host (optional - only for the hardware task; the RE task needs no hardware)
- Python (decryptor and tooling)
- Serial console (115200 8N1)

New to this? Install everything, step by step, for **Windows x64, Linux x64, or macOS arm64** in the companion course ([telescreen](#) → [docs/31-prerequisites-and-install.md](#)), then follow [docs/33-ghidra-nation-state-re.md](#).

Hints and Tips

1. **Trust the bytes, not the labels** - identify every partition by its magic.
2. **The environment is CRC-protected** - recompute the CRC before writing it back.
3. **The kernel is in a vendor container**, not a zImage - parse the header.
4. **JFFS2 uses crc32_le**, not zlib's CRC - this trips everyone the first time.
5. **The .cgi routes are internal** - only two are real files; the rest are code.
6. **Follow `system()`** - that is where user input becomes root.
7. **Nonce discipline** - if you reuse a GCM nonce in the lab, you have taught the wrong lesson; test for it.
8. **Document everything** - your analysis is the deliverable that survives the cell.